A Machine Learning Techniques

In this section, we briefly describe the ML methods that we used to predict the runtime required to compute explanations: linear and quadratic regression, regression trees, random forests, and neural networks. We use these ML techniques because they are commonly used in the literature to build models that predict the solving time of optimisation problems [23,18].

A.1 Linear Regression

LR is the most basic regression technique since it involves fitting a linear function $f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 \cdot x_1 + \cdots + w_n \cdot x_n$ to its input $\mathbf{x}$, where $\mathbf{w}$ are the parameters of the model. In our case, our inputs are the features characterising the UDSP instance $\mathbf{z}$, and the coefficients $\mathbf{w}$ weigh such features in our fitted model.

This technique aims to obtain the weights $\mathbf{w}$ that minimise the MSE, i.e., minimising $\sum_{i=1}^{N}(y_i - f_{\mathbf{w}}(\mathbf{x}))^2$, where y_i is the output to predict.

A.2 Quadratic Regression

Following Leyton-Brown et al. [23], we consider employing QR by constructing additional features that capture nonlinear interactions among the original ones, and subsequently fitting a linear model over this expanded feature set. Specifically, we augment the original features with all pairwise products, yielding a second-degree polynomial feature representation.

However, with such an approach, the number of features increases polynomially. This might cause problems during training when the number of features, and consequently the linear regression coefficients, exceeds the number of training instances. In addition, some features might be uninformative or strongly correlated to others. To mitigate the problem of having too many features, previous works in the literature use feature selection methods to reduce the number of features [18,31,32]. Therefore, we use forward selection to select the most relevant original features. This method iteratively chooses the best feature to add to the output feature set based on the cross-validation MSE of linear regression, and it stops when it has selected N features [12]. Then, we perform linear regression on all pairs of products from the selected feature set and its linear features. We use $N = 20$ as our parameter configuration for this method.

A.3 Regression Tree and Random Forest

Regression trees are tree-based regression models [5]. A RT is a tree where each non-leaf node is associated with an input feature. At each node, there's a binary split into two child nodes based on a splitting rule for the associated input feature and its possible values. A tree is built by recursively splitting the training dataset into subsets, each of which becomes a successor child. This process is repeated on each derived subset until the data in the leaf node share the same feature values.

Such a process can lead to *overfitting*, which can be mitigated with *pruning* methods that eliminate branches that do not improve the model's accuracy.

A RF is a method consisting of a set of regression trees [4]. Such trees can lead to different predictions because they are trained on different training sample sets or use different feature subsets to generate splitting rules. A RF prediction is computed by averaging the predictions of all the trees, hence preventing overfitting. The prediction quality improves as the number of trees, B, increases. We use $B = 100$ in the experiments.

A.4 Neural Networks

NNs are models inspired by the structure and function of the brain. They consist of layers of interconnected nodes, the so-called *neurons*, which process data to enable the network to learn patterns and make predictions or decisions. The MLP is one of the most basic NN architectures, where neurons are organised in layers. In detail, each neuron n_i receives k inputs $\mathbf{x_i} \in \mathbb{R}^k$ from neurons of the previous layer. Then, a neuron has associated a set of weights $\mathbf{w_i}$ and a bias term b_i which are used to aggregate the inputs such that $\mathbf{w_i} \cdot \mathbf{x_i} + b_i$. Finally, each neuron propagates the result to the neurons of the next layer through a nonlinear activation function. The weights of each neuron, $\mathbf{w_i} \in \mathbb{R}^k$, are adjusted so that they minimise a loss function, a process carried out using an optimisation algorithm.

We employ the well-known Python library TensorFlow [10] to implement two MLP models. We implement a model using one hidden layer of 128 neurons, which we call SL model. We use a second model using four hidden layers of 128 neurons, which we call DL model. Both models use ReLU activation functions and are trained using the "*Adam*" optimisation algorithm to minimise the MSE.

B Problem Instance Features

There is no automatic way to identify the features of a problem instance. To do so, we use specific problem features previously defined to predict the runtime of solving the WDP [23]. For the RCPSP, we use features defined to generate instances of the problem [20,19]. In addition, we add features that have a statistical relationship with the hardness of solving the problem instance.

B.1 WDP Features

Table 4 shows a list of the problem-specific features for the WDP. The first two features indicate the number of bids and goods of the instance, which in turn are the number of variables and constraints of the MILP encoding.

Secondly, we can define a bipartite graph representing the relations between bids (variables) and goods (constraints) of a WDP instance [23]. The nodes of the graphs are both the bids and the goods of the problem, and an edge indicates that a good is in a bid. We measure the node degree for each node and

Table 4: Specific features for the WDP instances defined by Leyton-Brown [23].

1-2 **Number of bids** (number of variables) and **number of goods** (number of constraints)

Bid-Good Graph Features

3-6 **Bid node degree statistics:** min, max, mean, and standard deviation of the bid node degrees.

7-10 **Good node degree statistics:** min, max, mean, and standard deviation of the good node degrees.

Bid Graph Features

11 **Edge Density:** number of edges in the BG divided by the number of edges in a complete graph (i.e., a *clique*).

12-15 **Node degree statistics:** min, max, mean, and standard deviation of the node degrees.

16-19 **Clustering Coefficient statistics.** A measure of "local cliqueness". For each node v_i, we calculate the number of edges between the neighbours N_i divided by the number of possible edges of the neighbours $k_i(k_i - 1)/2$, where k_i is the number of neighbours. Thus, the clustering coefficient of a node C_i is $\frac{2 \cdot |\{e_{jk} : v_j, v_k \in N_i, e_{jk} \in E\}}{k_i(k_i-1)}$. We record the min, max, mean, and standard deviation of the clustering coefficients.

20-22 **Minimum path length:** max, mean, and standard deviation of the minimum path length, over all pairs of bids.

23 **Ratio of the average clustering coefficient to the average minimum path length.**

24-27 **Node eccentricity statistics:** min, max, mean, and standard deviation of the node eccentricity. The node eccentricity is the length of the shortest path to its furthest node.

28 **Graph width.** A measure of how sparse the graph is, also known as "*graph degeneracy*". See [8].

LP-Based Features

29-31 L_1, L_2, and L_∞ norms of the integer slack vector.

Price-Based Features

32-35 **Bid price statistics:** min, max, mean, and standard deviation of the bid prices.

36-37 **Price per number of goods statistics:** mean and standard deviation of the bid prices per number of goods, i.e., $w_b/|G_b|$.

38-39 **Price per square root of the number of goods statistics:** mean and standard deviation of the bid prices per the square root of the number of goods, i.e., $w_b/\sqrt{|G_b|}$.

we compute the minimum, maximum, average, and standard deviation for each group of nodes. Intuitively, the average bid node degree indicates the average number of goods per bid. In contrast, the average good node degree indicates the average number of bids a given node is within.

Furthermore, we define a second graph, called the Bid Graph, which represents relations between bids [23]. An edge between bids indicates that they share at least one good. We measure different statistics to capture the structure of such a graph. Such features encompass the edge density, the node degree, the clustering coefficient, the minimum path length (between all pairs of nodes), the node eccentricity, and the graph width [8].

In addition, we compute metrics related to the LP relaxation of the problem instance. We do so since CPLEX uses the LP relaxation as a search instance [23]. Thus, we solve the LP relaxation of a WDP instance and we compute the integer slack vector of the solution, i.e., $min(|x_i|, |1 - x_i|)$. Such a vector gives us knowledge about the goodness of the initial non-integer solution. We measure the ℓ_1, ℓ_2, and ℓ_∞ norms of the integer slack vector.

Finally, we compute features related to the distribution of bid prices. In addition, we measure the price per good within a bid.

B.2 RCPSP Features

Table 5 shows a list of the problem-specific features for the RCPSP. The first two features indicate the number of variables and constraints of the MILP encoding. Similarly to the WDP, we can define a bipartite graph representing the relations

Table 5: Specific features for the RCPSP instances.

1-2	**Number of variables** and **number of constraints**.
Variable-Constraint Graph Features	
3-6	**Variable node degree statistics:** min, max, mean, and standard deviation of the variable node degrees.
7-10	**Constraint node degree statistics:** min, max, mean, and standard deviation of the constraint node degrees.
Activity Features	
11	**Number of activities.**
12-15	**Processing time statistics:** min, max, mean, and standard deviation of the activity processing times.
16-18	**Successors statistics:** max, mean, and standard deviation of the number of activity successors.
19-22	**Time range statistics.** Range of the activity time intervals, i.e., $LF_j - EF_j$. We record the min, max, mean, and standard deviation.
23-25	**Number of resources per activity statistics.** max, mean, and standard deviation of the number of different types of resources required per activity.
26-28	**Number of resource units per activity statistics.** max, mean, and standard deviation of the number of resource units required per activity.
Resource Features	
29	**Number of resources.**
30-33	**Resource availability statistics:** min, max, mean, and standard deviation of the number of resource units available each time period.
34	**Average resource factor:** the ratio of resource used. See [19] for details.
35	**Average resource strength:** the strength of resource constraints. See [19] for details.
Precedence Graph (PG) Features	
36	**PG Edge Density:** number of edges in the PG divided by the number of edges in a complete graph.
37-40	**PG Node degree statistics:** min, max, mean, and standard deviation of the node degrees.
41-43	**PG Minimum path length:** max, mean, and standard deviation of the minimum path length, over all pairs of activities.
44-47	**PG Node eccentricity statistics:** min, max, mean, and standard deviation of the node eccentricity.
Activity-Resource Graph (ARG) Features	
48	**ARG Edge Density:** number of edges in the ARG divided by the number of edges in a complete graph.
49-52	**ARG Node degree statistics:** min, max, mean, and standard deviation of the node degrees.
53-54	**ARG Clustering Coefficient statistics.** We record the mean and the standard deviation of the clustering coefficients.
55-57	**ARG Minimum path length:** max, mean, and standard deviation of the minimum path length, over all pairs of nodes.
58	**ARG Ratio of the average clustering coefficient to the average minimum path length.**
59-62	**ARG Node eccentricity statistics:** min, max, mean, and standard deviation of the node eccentricity.
LP-Based Features	
63-65	L_1, L_2, and L_∞ norms of the integer slack vector.

between variables and constraints of a RCPSP instance, and we extract the same metrics for such a problem.

Then, we define features to characterise the activities of a problem instance. We measure statistics in relation to their processing time, the number of successors, the time range interval an activity j can be completed, i.e., $LF_j - EF_j$, as well as the total number of resources and resource units an activity needs to be completed. In addition, we gather information about the resources. We measure the number of resource types, the number of resource units available for each time period, as well as the average resource factor and the average resource strength. Both metrics are used to generate different random instances of the RCPSP [19,20].

Likewise, in the case of the WDP, we define a Precedence Graph. We do so to gather information about the precedence relations in the problem instance. In such a graph, the nodes of the graphs are the activities of the problem instance. The edges between activities indicate a precedence relationship between them. Following the case of the WDP, we measure statistics such as the edge density,

Table 6: Set of possible user questions, query constraints and constraint encodings for the Resource-Constrained Project Scheduling Problem formalisation.

	Question	**Query constraints**	**Constraint encoding (C_Q)**
Q1.	Why is activity j completed at time t?	**veto** j complete at time t	$x_{j,t} = 0$
Q2.	Why is activity j **not** completed at time t?	**enforce** j complete at time t	$x_{j,t} = 1$
Q3.	Why is activity j **not** completed before time t?	**enforce** j complete before time t	$\sum_{t'=EF_j}^{t-1} x_{j,t'} = 1$
Q4.	Why is activity j **not** completed after time t?	**enforce** j to complete after t	$\sum_{t'=t+1}^{LF_j} x_{j,t'} = 1$
Q5.	Why is the group of activities A completed at t?	**veto** to complete $j \in A$ at time t	$\sum_{j \in A} x_{j,t} < \lvert A \rvert$
Q6.	Why is the group of activities A **not** completed at t?	**enforce** to complete all of them at time t	$\sum_{j \in A} x_{j,t} = \lvert A \rvert$
Q7.	Why is activity j completed at t instead of t'?	**enforce** complete j at t' and **veto** j at t	$x_{j,t'} = 1$, $x_{j,t} = 0$
Q8.	Why is activity j completed at t instead of j'?	**enforce** j' and **veto** j complete at time t	$x_{j',t} = 1$, $x_{j,t} = 0$

the node degree, the minimum path length (between all pairs of nodes), and the node eccentricity.

Furthermore, we define a second graph, called the Activity-Resource Graph, which represents relations between activities that share resources. Each node is an activity, and an edge between activities indicates that such activities share at least one resource in a given time period. In other words, an edge represents whether two activities are in the scope of a resource constraint. Similarly to the Precedence Graph, we measure the following statistics: the edge density, the node degree, the clustering coefficient, the minimum path length (between all pairs of nodes), and the node eccentricity.

Finally, as we implemented in the WDP, we compute metrics related to the LP relaxation of the problem instance. We measure the ℓ_1, ℓ_2, and ℓ_∞ norms of the integer slack vector.

C Query Examples

C.1 Resource-Constrained Project Scheduling Problem

Table 6 lists possible queries for the RCPSP. The table also shows the translations of queries into (linear) query constraints.

In the case of the RCPSP, for example, the query "*Why is activity j scheduled to be completed at time t?*" translates into **vetoing** the completion of activity j at time t (i.e., $x_{j,t} = 0$) to enforce an alternative solution to the optimal solution.

Table 7: Set of possible queries, the query constraints and the constraint encoding of the Winner Determination Problem formalisation.

Question	Query constraints	Constraint encoding, C_Q
Q1. Why is bid b selected?	**veto** b	$x_b = 0$
Q2. Why is bid b **not** selected?	**enforce** b	$x_b = 1$
Q3. Why is bid b selected instead of bid b'?	**enforce** b', **veto** b	$x_{b'} = 1, x_b = 0$
Q4. Why is good g selected?	**veto** g	$\sum_{b \in B \mid g \in G_b} x_b = 0$
Q5. Why is good g **not** selected?	**enforce** g	$\sum_{b \in B \mid g \in G_b} x_b = 1$
Q6. Why is good g selected instead of good g'?	**enforce** g', **veto** g	$\sum_{b \in B \mid g' \in G_b} x_b = 1$, $\sum_{b \in B \mid g \in G_b} x_b = 0$
Q7. Why are goods g and g' in the same bid?	**veto** g and g' in the same selected bid	$\sum_{b \in B \mid g', g \in G_b} x_b = 0$
Q8. Why are goods g and g' **not** in the same bid?	**enforce** g and g' in the same bid	$\sum_{b \in B \mid g', g \in G_b} x_b = 1$

Notice that this query refers to a **single assignment** occurring in the optimal solution.

Additionally, to comply with the query "*Why is the group of activities A scheduled to be completed at time t?*" we must **veto** the completion of all activities in A at t (i.e., $\sum_{j \in A} x_{j,t} = 0$). In this case, the query refers to the schedule of multiple activities, i.e., **multiple assignments**, in the optimal solution. Furthermore, when addressing the query "*Why is activity j **not** scheduled to be completed at time t?*" we must **enforce** the hypothetical scheduling of activity j, which the current scheduling disregards.

Moreover, to answer the query "*Why is activity j scheduled to be completed at time t instead of activity j'?*", we must: (1) enforce activity j' ; and (2) veto j at time t. Thus, we consider combinations of **enforce** and **veto** constraints to answer more specific queries about value assignments that did and did not occur.

C.2 Winner Determination Problem

Table 7 presents the set of possible queries for the WDP, together with their translation into (linear) query constraints.

In the context of the WDP, the query "*Why is bid b selected?*" is addressed by **vetoing** bid b, that is, by adding the constraint $x_b = 0$ to rule out the current optimal choice and enforce an alternative solution. This query refers to a **single assignment** in the optimal solution, namely the selection of a specific bid.

Similarly, the query *Why is bid b **not** selected?*" is handled by **enforcing** the selection of b (i.e., $x_b = 1$), thereby imposing a hypothetical alternative that contradicts the optimal assignment. Another example is query Q3, "*Why is bid*

Table 8: Solving time prediction results for the different models and different problem instances. We report 10-fold cross-validation RMSE and MAE (lower values are better). Bold indicates the (significantly) best-performing model.

Problem Domain	RMSE						MAE					
	LR	QR	RT	RF	SL	DL	LR	QR	RT	RF	SL	DL
WDP	0.27	0.20	0.25	0.17	**0.10**	0.14	0.18	0.35	0.13	0.10	**0.08**	0.12
RCPSP	0.49	**0.41**	0.55	**0.41**	0.45	0.56	0.36	0.52	0.36	**0.27**	0.35	0.48

b selected instead of bid b′?", which requires a combination of constraints: we **enforce** b' and simultaneously **veto** b. Hence, these queries involve combinations of enforce and veto constraints to compare assignments that did and did not occur in the optimal solution.

Beyond individual bids, some queries concern **goods** rather than bids. For instance, the query *Why is good g selected?*" is translated into **vetoing** all bids that contain g, that is, imposing $\sum_{b \in B | g \in G_b} x_b = 0$. In contrast, the query *Why is good g **not** selected?*" is handled by **enforcing** that exactly one bid containing g be selected, i.e., $\sum_{b \in B | g \in G_b} x_b = 1$. These queries refer to **multiple assignments**, since the selection of a good depends on multiple bids which include it.

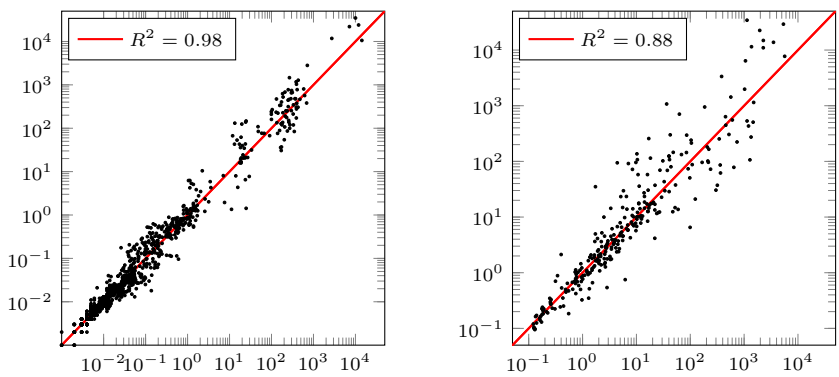
Overall, as in the RCPSP, all queries in the WDP can be encoded through combinations of **enforce** and **veto** constraints, applied either to single decision variables (bids) or to aggregated sets of variables (goods and their co-occurrence in bids).

D Solving Time Prediction

In this section, we show the results of applying the methodology of previous works on runtime prediction [25] to predict the solving time of the WDP and the RCPSP.

Following a well-known methodology in the literature [18], we predict solving times to evaluate whether the problem instance features properly characterise the optimisation problem's hardness. In Table 8, we show the results of our predictions. We show that both the RMSE and the MAE are low for all the prediction models, even for simple models such as LR, since the errors are $\ll 1$. This indicates that the runtime prediction is of the same order of magnitude as the ground-truth value. In detail, we show that the "*shallow learning*" model achieves the best performance on the WDP, closely followed by the deep learning model and the RF. For the RCPSP, the best models are QR with feature selection and the RF (although the RF has a lower MAE).

Figure 6 shows the scatter plots and the adjusted R^2 of the solving time predictions of RF on test data for both MIP problems in Figures 6a and 6b. Notice that both Regression Coefficients (R^2) are close to 1.



(a) Solving WDP (b) Solving RCPSP

Fig. 6: Solving time predictions on previously unseen test instances for Random Forest (best performing model). The $x-$axis denotes the runtime prediction, and the $y-$axis denotes the real runtime. Each data point represents one instance. We show the Coefficient of Determination (R^2), which measures how well the model predicts runtime (1 indicates a perfect fit).